

FRED TALKS

20th May 2026

CONTENTS

LLM Evaluation: Practical Tips at Booking.com

GEORGE CHOULIARAS · 2734 WORDS

Can agents replace the search stack?

DOUG TURNBULL · 1252 WORDS

10 Lessons for Agentic Coding

DREW BREUNIG · 638 WORDS

The Bag-of-Documents Model for Query Understanding and
Retrieval

DANIEL TUNKELANG · 78 WORDS

LLM Evaluation: Practical Tips at Booking.com

GEORGE CHOULIARAS · 13 SEP 2025 · [SOURCE](#)

Lessons learned from 1 year of Judge-LLM Development

By

Zeno Belligoli

,

Antonio Castelli

,

George Chouliaras



The increasing adoption of Large Language Models (LLMs) across various industries has made evaluating LLM-powered applications (also called Generative AI applications) a critical necessity.

LLMs are pre-trained on a vast amount of text and can therefore be used to perform a wide range of tasks based on a natural language input called **prompt**. These tasks range from extracting information, answering complex questions, summarizing documents, generating creative content, translating languages, writing code etc.

However, unlike traditional Machine Learning (ML) models, the nature of LLMs brings inherent challenges that must be carefully considered:

- LLMs can **hallucinate**, i.e. can generate outputs which are factually incorrect or nonsensical while presenting them with high confidence
- LLMs can fail to **follow the instructions** specified in the prompt no matter how detailed they sound to a human reader
- Usage of LLMs has a **non-negligible cost** whether the model is open-source and served in-house or closed-source and accessed via a paid API.

To maximize the potential of Generative AI (GenAI) applications and mitigate the risks associated with them, we built a framework capable of thoroughly evaluating the performance of an LLM on a specific task in a nearly automated way. This framework is based on the concept of **LLM-as-judge**, i.e. on the usage of a more powerful LLM to evaluate the “target” LLM.

The LLM-as-a-judge framework

The main difference between the evaluation of traditional ML models and LLMs is that in most of the cases we are dealing with generative tasks for which either no single ground truth exists or is hard / expensive to obtain. When we ask an LLM to write a description of a property, or to summarize a long text, we don’t really have an unambiguous reference that we can use for comparison. Moreover we can’t get this at scale for different topics or subjects.

Therefore the measurement of text attributes like clarity, readability, toxicity or factual accuracy becomes more challenging. It would be possible, in theory, to employ human experts to review every generation and provide an estimation of the metric under study. However, this process would be time consuming and expensive to be **practically infeasible**.

The LLM-as-judge approach, requires human involvement **only once** (unless the production distribution changes), to carefully annotate the so-called **golden dataset** which must be large enough to be representative of the data distribution in production (the best practices for golden dataset creation are described in the next section).

Once we have the golden dataset with labels, we can prompt (or fine-tune) an LLM to replicate human judgement as much as possible. Once this is achieved, (e.g. when the agreement between the judge LLM and the human annotation has an accuracy above a certain threshold) the judge LLM can be employed to score the predictions of a second LLM (*target LLM*) on other datasets.

This allows the continuous monitoring of the performance of a GenAI application in production in a scalable way with minimal human involvement.

Since the judge-LLM solves a classification task (e.g. is the text clear or not? is the text factually accurate or not?) getting golden labels is much easier. Hence, we can evaluate its performance using standard metrics like accuracy, f1-score, precision or recall, to monitor the performance of the target LLM. The process of judge-LLM development and usage is depicted in Figure 1.

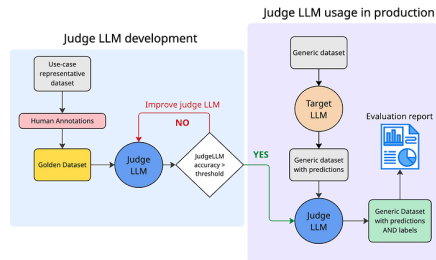


Figure 1: Judge-LLM development and deployment cycle. The left box (light-blue) shows the judge-LLM development phase. This phase always starts with a golden dataset with labels provided by a fully trusted source (e.g. human experts). A base LLM model is prompted or fine-tuned to imitate human label scoring. When the judge-LLM reaches a certain accuracy with respect to the golden labels, it can be deployed in production (right purple box) to score any other similar dataset, for example to evaluate the output of a target LLM. An evaluation report can be produced using the predictions of the target LLM and the label provided by the Judge LLM.

Golden Dataset Creation

The main aim of a golden dataset is to **accurately assess the ability** of a judge-LLM, to evaluate a production LLM in a particular evaluation metric. Due to this, a **high quality** golden dataset should meet the following criteria:

- It should represent the *production distribution*, since we want the Judge LLM evaluation to be as close to production as possible.
- It should include *golden* labels, e.g. labels with high quality. This can be proxied by high inter-annotator agreement among the annotators of the labels.

Low inter-annotator agreement usually indicates ambiguity in the metric definition and should be resolved by calibration among the annotators and by adding clarifications in the definition until the agreement is acceptable.

Since text attributes often do not have an objective and unique true value, achieving high-quality annotations is not a trivial task. To make fair comparisons between different versions of a GenAI application it is of utmost importance to have a **standardized annotation protocol** which is rigorously followed. Below we provide both a **basic** and an **advanced** annotation protocol for creating the golden dataset.

Basic Annotation Protocol

The basic annotation protocol can be used when only a single annotator is available.

1. Metric Definition

Define the metric definition with the business owner in the most unambiguous way possible.

- Write a **clear** definition of the metric as objective as possible. Prefer **binary** metrics or categorical metrics with a few classes, since LLMs struggle with continuous scores. If you're forced to use a continuous metric, consider binning into a few coarse ordinal values e.g. 'high', 'medium', 'low'.
- Write **annotation instructions**, using the metric definition. These are the instructions a human should follow in order to provide the ground truth label. Describe the task **clearly** and **objectively**. Avoid vague words (such as major, minor, a few). Provide scored examples and include edge cases. Specify what tools the annotators can and cannot use (Search engines, ChatGPT, etc).

2. Pilot Annotation

Collect a few examples (~50) and annotate them according to the aforementioned metric definition. The annotators should carefully follow the instructions provided, without bringing any previous knowledge or bias on the task. The annotators at this stage can be selected from business owners or from other technical people involved. The aim of this step is to ensure that we have a clear metric definition and that the annotations are aligned with it.

Once the annotations are done, it is recommended that they are **reviewed** by a pool of domain experts to assess if they align with the guidelines. If issues or disagreements are found, they should be resolved, the metric definition should be updated and the pilot annotation should be repeated until full consensus is reached.

3. Full Annotation by a single annotator

Once the annotation of the first small sample during the pilot annotation is finalized and reviewed, the examples can be shared to the annotator as guidance in order to annotate a full dataset. The full dataset should consist of **ideally 500- 1000 examples** at least. This is because part of the dataset will be used for tuning the judge-LLM (validation set) and another part as a holdout set in order to evaluate its performance.

4. Quality Review

Get George Chouliaras's stories in your inbox

Join Medium for free to get updates from this writer.

A subject matter expert reviews a **representative** sample (~10%) of the full annotation. If a significant percentage (> 10%) of the labels in the sample are incorrect, annotation should be repeated. The quality thresholds can be adjusted according to the organization's needs and should be communicated and agreed upon with the annotators before the full annotation.

Advanced Annotation Protocol

The advanced annotation protocol requires more effort but also guarantees higher quality annotations than the basic one.

1. Metric Definition

Same as in the basic protocol.

2. Pilot Annotation

Same as in the basic protocol.

3. Full Annotation by multiple annotators

Instead of having a single human annotator providing the label for a given example, we have multiple human annotators (3+) providing a label for each example.

Depending on the use case, an aggregation function must be defined to obtain the final label.

The aggregation function should take an array of labels and return a single one. In principle any

aggregation function can be used. If labels can be converted into integers and ordered any function like min, max, or average can be used. If labels have no orders (e.g. geographical location) majority vote can be used.

Examples where the annotators **disagree** can be treated in multiple ways, depending on what is more appropriate for the specific problem.

a) Add a “not sure” class: map to the “not sure” class all examples without 100% consensus

b) Add a weight < 1 : For all examples without 100% consensus select the majority class among the annotated ones (a random one if there is no majority) and add a weight to the example equal to $n_selected_class_votes / n_total_votes$. This approach will penalize the more uncertain examples in the aggregated performance.

c) Discard the ambiguous examples: For some use cases it might make sense to simply remove all the examples where there is no 100% agreement.

4. Quality Review

Same as in the basic protocol.

How to develop a judge-LLM

Once the golden dataset is ready, you can develop the judge-LLM. Note that you can already do this step once the sample dataset from the pilot annotation is ready (with ~50 samples) to create a first version of the judge-LLM and score a few examples to identify any early issues. However for proper evaluation and tuning of the judge-LLM you need a large dataset of 500–1000 examples. In this article we share how to create a prompt-based judge-LLM for simplicity. You can always fine-tune a judge-LLM but this won't be covered here. To develop the judge-LLM you follow the **iterative process** detailed below, which is the standard process for manual prompt engineering.

1. **Split the golden dataset in validation and test sets:** A standard split is 50/50 but the final decision is on the model owner. The validation set is used to compute the prompt performance and to find error patterns. The test set is used to check for evaluating generalization and overfitting.
2. **Choose a strong LLM:** We typically select powerful models such as **GPT-4.1**, or **Claude 4.0 Sonnet** as the backbone for our judge-LLM. These models serve two roles:

a) As a sanity check: Given a high-quality dataset and a well-defined task, a strong LLM should perform well if the prompt is reasonable.

b) To estimate an upper bound on achievable performance. While this is not a strict bound (there are cases where smaller models with better prompts outperform stronger ones) it provides a useful reference point.

3. Write the prompt of the judge-LLM: Use the metric definition and include scoring instructions and output format (can be json/string/else). Few shot examples can also be included but ensure that no examples from the golden dataset are provided to avoid overfitting. We also normally add chain-of-thought prompting (CoT) to promote reasoning and explainability. To build the judge-LLM you can use any framework that supports custom LLM-based metrics, such as DeepEval's G-Eval metric or Arize Phoenix.

4. Evaluate the performance on the validation set: The evaluation metric depends on the type of the task and the class imbalance. For classification tasks we generally recommend macro f1-score while for continuous metric you can use mean squared error.

5. Perform error analysis and update the prompt

We analyze the model's mistakes on the validation set to identify recurring issues or edge cases. Based on these insights, we revise the prompt to better align it with the desired behavior. We then repeat steps 3, 4 and 5. Typically, we are satisfied once we reach a value of the evaluation metric above a certain threshold agreed with the business stakeholders.

6. Evaluate the performance on the test set: This will give the unbiased performance score of the judge-LLM. It is not recommended to change the prompt in order to improve the performance on the test set, as this might end up in overfitting on this dataset.

After prompt engineering is completed with a strong model, we repeat the same process using a **weaker (and more cost-efficient) model**, aiming to match the performance of the stronger version as closely as possible. In practice, we use the **strong-model judge-LLM during AI system development**, where quality is critical, and **the weaker-model version for large-scale monitoring** of system performance in production environments. An example of such monitoring is provided in Figure 2.



Figure 2: The figure shows an example dashboard used to monitor the quality of an LLM application, based on judge-LLM metrics. Metric values can also be combined to get aggregated scores. In this particular example we show the trends for Entity Extraction accuracy, LLM-instruction-following accuracy, frustration of users, context relevance, LLM-location-resolving accuracy and LLM-topic-conversation-extraction accuracy. From the plot it is immediate to quickly spot issues connected to instruction-following and topic-extraction allowing for prompt mitigation actions. An automated system is set-up to alert the application owners whenever an anomaly is detected for any of the relevant metrics.

Future directions

Pointwise vs. Comparative Judges

The vast majority of our judge-LLMs are **pointwise judges**, which assign an absolute score to each response independently. Pointwise judges are particularly useful because they can serve **dual purposes**:

- Ranking system outputs during development.
- Monitoring performance in production, where only one system's response is typically available.

However, **comparative (pair-wise)** judges (which compare two responses and select the better one) tend to offer **stronger ranking signals**. It is often easier for a model to decide which of two answers is better than to assign calibrated absolute scores. For this reason, comparative judges can be more effective during the AI system development phase. We are actively exploring methods to **develop both pointwise and comparative judges with minimal extra overhead**, ideally reusing most of the dataset and prompt engineering efforts.

Automatic judge-LLM development

Prompt engineering remains a **manual, iterative, and time-intensive process**, typically taking anywhere from **one day to a full week** depending on the complexity of the task. To streamline this, we have developed an **automated prompt engineering pipeline** inspired by [DeepMind's OPRO](#) that, in essence, automates **steps 3–5** of the process. At the end of the process it is always important that the human inspects the final prompt to ensure that it doesn't contain use case specific examples or rules that might undermine generalizability.

Another bottleneck in the judge-LLM development process is that of data annotation. Annotating a good dataset requires substantial effort from multiple annotators and can take from some days to weeks, depending on the complexity of the task. In this context, one can leverage the ability of LLMs to generate realistic text to create **synthetic data**. This is particularly helpful and achievable for classification tasks, whereby the LLM only needs to generate the input X , conditioned on a certain class Y . Substantial challenges in this area lie in the generation of diverse and realistic text and conversations and in the quality evaluation of the synthetic data, that is, how do we know that our synthetic data is realistic, diverse and of high-quality? We are actively working on this area and are looking forward to sharing more with you in a future blog post.

Evaluating LLM agents

We did not cover LLM-based **agent evaluation** in this article as it is a topic on its own. This is because evaluating agents often requires assessing long-term goal completion, reasoning over multi-step interactions, handling complex tool use and effective use of planning and memory. These are challenges that go beyond the evaluation of isolated model outputs and typically involve different methodologies, benchmarks, and infrastructure. We will cover this topic in a separate blog post.

Key takeaways

Evaluating generative tasks is inherently complex due to the absence of clear ground truth data. The LLM-as-a-judge framework offers an efficient method to scale this evaluation process in an automated way. A high-quality judge-LLM is built upon a “golden dataset” with reliable labels, which necessitates a rigorous annotation protocol; we have outlined both a basic and an advanced version to achieve this. This judge-LLM can then be optimized through manual, iterative prompt engineering or via automated methods. Future directions in this field include creating golden datasets with synthetic data and developing evaluation methods for LLM-based agents.

Can agents replace the search stack?

DOUG TURNBULL · 28 APR 2026 · [SOURCE](#)

How is search implemented where you work? Probably as a complex set of capabilities on top of retrieval. Our search APIs understand queries, call backend search systems, and finally rerank results.

But if we had an agent in the loop, would we need all that? Could we replace search backends with an agent? After all, an agent understands user requests, calls retrieval tools, and evaluates relevance on its own. We see ChatGPT do this all the time, why can't our search bar?

In other words, if you give it a basic BM25 backend to an agent, could it take the Search API's job?

Surprisingly good results with simple tool usage

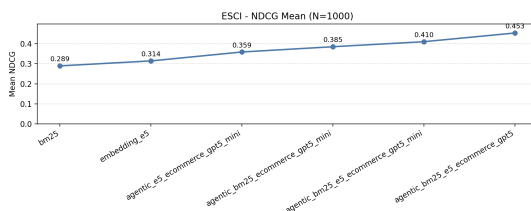
Well I've measured initial findings (a bit deeper than in the past). I gave agents basic search tools and got the popcorn out to watch.

On Amazon ESCI, we see increasing performance with slightly better retrievers + LLMs.

First, we start with a bm25+e5_embedding baseline. They achieve 0.289/0.314 NDCG respectively.

Then we measure three agentic experiments. Each with GPT-5-mini driving different combinations of search tools. We see an agent with e5 embeddings get 0.359 and with BM25 0.385. With both we get 0.410.

GPT-5 with both tools gets an NDCG of 0.453.

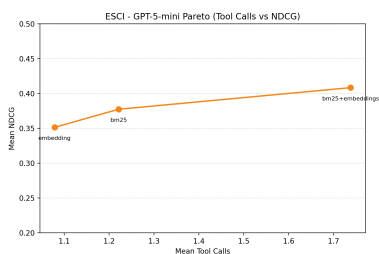


What's amazing about this? I've barely lifted a finger. I just setup a stock LLM with some search tools. Nothing here has been fit to my data. A $0.289 \rightarrow 0.453$ jump in quality depends only on model quality + simple retrievers.

If an agent could be fast enough, is there a future here we build search with agents and not APIs? A world where agents reason about queries to show users the best results?

But, agents call search approximately... once?

What kind of ingenious strategies are these agents employing? It turns out, for the most part, they're calling search roughly once per search tool. You can see this in the graph.



When you look at reasoning traces, you see the agent interpret the search, issue a single query, retrieve up to the max of 20 results, then rank them.

Occasionally, very rarely, with just BM25, you'll see two simultaneous searches. For example, "PVC coupler" shows first a BM25 search for "pvc coupler", the agent, disappointed with results, issues a follow up query (PVC pipe coupler):

Calling `search_bm25` with args `keywords='pvc coupler'` `top_k=20`

```
**Searching for PVC couplers** I noticed that the search results mainly returned RJ45 couplers, which are for network cables, not what the user meant. They're probably asking about PVC pipe couplers used in plumbing. The current search tool isn't yielding entries for PVC pipe couplers specifically. I saw mentions of PVC in descriptions, but it's clearer to use broader terms. I'll refine my search. I need to focus on terms like "PVC pipe coupler" or "coupling" to get better results, so I'll try that next.
```

Calling `search_bm25` with args `keywords='PVC pipe coupler'` `top_k=20`

You also see cases where the agent uses BM25 to issue two queries simultaneously, anticipating inconsistent results:

```
Calling search_bm25 with args keywords='AN10 oil catch can no filter'  
Calling search_bm25 with args keywords='AN10 oil catch can without br
```

There’s something about keyword search, it seems, that nudges the agent to try more queries. The agent can interpret results, understand why they didn’t match, and take next steps. Keyword search has cause-and-effect (keywords matched / or didn’t). Agents can react to that and improve.

Encouraging exploration improves further

I want agent exploration to be the norm, not the exception.

We haven’t changed a thing about the harness. We could steer agents to search more effectively. What happens with a few small changes to our setup:

- **Require more tool calls** - we ask the agent to keep trying if its made fewer than the required number of tool calls (ie 4). We say something encouraging like “wow these are great, but I bet you can do better!”
- **Require the agent use diverse retrieval queries** - to ensure the agent truly explores, we reject queries too similar to those issued by the agent previously for this search.

Within the GPT-5-mini examples, we see incremental improvements.

strategy	model	mean	median
<u>Two tools</u>	gpt-5-mini	0.4101	0.3743
<u>Require 4 calls, disallow duplicates</u>	gpt-5-mini	0.4290	0.3948
<u>Require 4 calls, disallow queries > 0.9 sim to past queries</u>	gpt-5-mini	0.4308	0.4258

That’s nice, it approaches the GPT-5 benchmark from before, but with a smaller model. Albeit with more tool calls.

What about agentic search models (ie SID-1)?

Another source of hope comes from model providers. Teams like [SID](#) have agentic search models that act as drop-in replacements for your RAG stack. These self-hosted agents translate your company's simple retrieval tools into relevant results. That's basically replacing a search API with an agent.

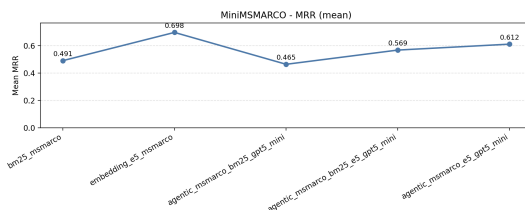
It's important to have a different model. Frontier lab LLMs think of "search" as web search. So they issue a query once, trust the results, and move on. That's valuable because they're reasoning about the user's problem - not search relevance.

But a model like SID-1 might be trained to reason about relevance using less sophisticated retrievers. They know to explore more of the corpus and reason efficiently about results. As a self-hosted subagent, focused only on retrieving results that satisfy the search, they leave the main LLM focused on the user's task.

Still, to be smaller, they must be more focused. One wonders if in the future we'll have agentic search LLMs tailored to different domains. One good at e-commerce. Another good at document retrieval, etc. The tradeoffs look like embedding models: we must choose the model based on its training data. We understand we can drift away from the model's sweet spot.

"Finding" different than Deep Research**

Where does this break down? Change to passage dataset, and there's no improvement by letting an agent drive tools. You can't beat the embedding model.



MiniMARCO MRR

The e5 model here almost certainly has MSMarco in its training set. In this case **the retriever knows best**. The Agents own evaluation of results hits a wall.

We cross a threshold where the LLMs and users diverge in their relevance evaluations. An 80% accurate LLM as a judge isn't that useful, actually. Often that 80% holds the easy cases. It's the last 20% that's hard.

In other words, LLMs don't know everything. Search when the LLM "knows" vs "doesn't know" are different problems.

So far we've focused on finding **things** (products, jobs, etc). But adding knowledge the agent doesn't have? That's fundamentally a different problem. In the case above, the embedding adds knowledge to the agent. When an agent encounters a question it can't answer in that last 20%, it needs search to be an oracle. It's not finding things anymore, it's filling in its own gaps.

That's how ChatGPT treats web search. Ask about a current event past the training cutoff, you'll see it in action.

To me this last 20% says everything about search. It's where the really hard search problems exist. You either:

- Build a model that learns your domain - ie a SID-1 for your data + queries
- Build a thicker / smarter retrieval function, as in classic search. It continues to solve search the old fashioned way - with new techniques like late interaction helping

So in the end, can an agent replace a search API? I'd argue there's still a tremendous amount of skill needed to build a useful retrieval function. We can build a harness so strong even grep / BM25 mostly works. But we pass a threshold with diminishing returns.

Enjoy software Doug in training course form!

Starting May 18!



I hope you join me at [Cheat at Search with Agents](#) to learn use agents in search. build better RAG and use LLMs in query understanding.

10 Lessons for Agentic Coding

DREW BREUNIG · 08 MAY 2026 · [SOURCE](#)

What should we do when code is cheap?



Lately, this blog has featured a lot of writing about agentic coding. Frontier models are *really* good at coding these days, much better than they are at other tasks. Coding with agents feels like a preview of the future, a playground for seeing how far we can push agent capabilities. It's invigorating, rewarding, and deeply *weird*.

I've been keeping a running list of tips for agentic coding: guidelines or rules one might give to someone just getting started with Codex, Claude Code, Pi, or any other agent. Ideally each tip is generalizable guidance, relevant to any agentic programming. I'm also looking for durable lessons that will stick around as models and harnesses improve.

Below is my current list: *10 Lessons for Agentic Coding*. Ten's a nice round number; a good time to put this out there.

To be clear: I take credit only for honing and compiling these guidelines. As [Kshetrajna Raghavan](#) said to me today, "It's *crazy* how we're all converging on similar lessons."

(If you think I've missed anything below, please reach out!)

10 Lessons for Agentic Coding

1. **Implement to learn.** You can go far with Spec-Driven Development, but the act of writing code surfaces decisions you hadn't considered and makes your spec better. When code is cheap, implement to learn.

2. **Rebuild often.** Implement early and *often* to learn more. Fork and recode crazy thought experiments. Find out how far you can take feature. Of course, you want to iterate and compound your efforts, but cheap code means you can reconnoiter and reinvent in ways you never could.
3. **Invest in end-to-end tests.** When we can reinvent our code cheaply, we should spend time writing tests that measure our product's *functions*, not *how* it performs them. We want behavioral contracts that grant us the freedom to rebuild and reimplement.
4. **Document intent.** Tests detail our goals while code encodes our methods, but neither captures the *why*. Your intent motivates your decisions, and persisting it alongside the code helps you and your agent compound those decisions in a consistent direction.
5. **Keep your specs in sync.** Update your specs, the markdown files containing your goals and plans, as you advance your code and your tests. Treating your spec as a frozen artifact written before work begins, you'll fail to capture learnings during implementation. Keeping it current lets it constantly inform your and your agents' choices, and makes frequent rebuilds easier.
6. **Find the hard stuff.** Work on a project long enough and things will stop being easy. You'll speed through the boilerplate work, the obvious design decisions, and start hitting the ugly, difficult work: intuitive design, performance, security, resilience, and systemic architecture. Anyone can vibe the easy stuff. The hard work is where the value is. Find it and dig in.
7. **Automate everything that's easy.** To spend more time on the hard stuff, minimize the time you spend on easy things. Distill learnings into skills, build loops, automate code reviews, and let your tools compound. But careful: don't get stuck in a Mystery House.
8. **Develop your taste.** When code arrives fast but feedback doesn't, the only source of feedback that keeps up is your own. The better you know your domain, your users, and their problems, the further you can go without checking in.
9. **Agents amplify experience.** Talented developers underestimate how much intuition they bring to their prompts: the right terms, the right framing, and the right level of specificity. If you know your stack, you can save countless cycles during both implementation and debugging, and cut down needless agent exploration. Pair technical expertise coupled with great taste for an unbeatable advantage.
10. **Code is cheap, but maintenance, support, and security aren't.** Agentic code is "free as in puppies." Support isn't cheap and neither is security. Build fast, but mind the maintenance you're adopting.

The Bag-of-Documents Model for Query Understanding and Retrieval

DANIEL TUNKELANG · 19 MAY 2026 · [SOURCE](#)

Today I had the pleasure of presenting my [recent work on the bag-of-documents model](#) as part of a series of talks hosted by [Trey Grainger](#) and [Doug Turnbull](#) in the context of their [AI-powered search classes](#).

You can find the [slides](#) below. I also encourage you to check out the [bag-of-documents Github repository](#), the [dataset and model on Hugging Face](#), as well as the live [Amazon demo](#) and [Best Buy demo](#). Enjoy!

<https://medium.com/media/12dd1c01b9dd78114555f2c6ed60a2fe/href>